

A Replication and Analysis of the Pixel Stealing Attack

Jeremy Sheng, Joy Chen, Maddie Dreher, Diya Shrawagi
The University of Texas at Austin
{js224768,cc75925,mbd2325,ds54998}@eid.utexas.edu

Abstract

Modern web browsers enforce isolation between websites so that a malicious webpage should not be able to directly read the contents of another origin. However, side-channel attacks can sometimes infer information indirectly through shared browser and hardware resources. In this project, we propose to study and reproduce a pixel-stealing attack in a controlled environment. We will create a malicious webpage and a separate victim webpage containing known visual information, then investigate whether rendering-related side channels can be used to recover information from the victim page while the user remains on the malicious site. Our primary goal is to determine whether such an attack can be reproduced, how accurately information can be recovered, and how long recovery takes. As a stretch goal, we will investigate whether partial recovery progress can be saved and resumed if the victim later revisits the malicious webpage.

1 Introduction

Browsers are designed around the same-origin policy, which prevents one website from directly inspecting the contents of another website. Despite this isolation, webpages still share underlying resources such as the CPU, GPU, caches, and browser rendering system. Differences in how these resources behave while processing particular visual content can potentially leak information through a side channel.

Our project will investigate whether we can reproduce the general idea of a pixel-stealing attack within a controlled experimental environment. Rather than attacking real websites or user data, we will create a synthetic victim webpage whose contents are known to us. A separate malicious webpage will attempt to infer those contents indirectly. We are particularly interested in understanding the relationship between attack duration and recovery quality.

2 Background

A side-channel attack recovers information indirectly by observing effects such as execution time, memory access patterns, or cache behavior rather than directly reading protected data.

Modern browsers render webpages by processing HTML/CSS and sending graphical work through the browser’s rendering pipeline, often using both the CPU and GPU. During this process, different webpages may share hardware resources such as processor caches and GPU memory.

Pixel-stealing attacks combine these ideas by observing rendering-related timing or cache effects to infer properties of visual content that the attacker is not allowed to directly access.

3 Threat Model

We assume that an attacker controls a malicious website and convinces a user to visit it. The attacker does not have direct access to the victim website’s DOM, memory, network traffic, or authentication information. Instead, the attacker can only execute ordinary browser code from the malicious origin and observe side-channel measurements exposed through shared browser or hardware behavior.

For our experiments, the victim will be a webpage that we control and populate with known content such as simple shapes, colors, text, or a small grid of pixels. This gives us ground truth for evaluating the attack.

4 Our Approach

We will first construct a controlled experimental setup consisting of two origins: a malicious webpage and a victim webpage. The victim webpage will display known visual information that we attempt to recover.

We will then reproduce a rendering or hardware side channel based on techniques described in existing pixel-stealing research. At a high level, the malicious webpage will repeatedly perform rendering measurements whose behavior may depend on visual properties of the victim content. We will collect these measurements over time and determine whether statistical differences can distinguish between different victim pixel values or visual patterns.

We will begin with a deliberately simple victim, such as distinguishing between two colors or recovering a small binary image. If this succeeds, we will gradually increase the complexity of the recovered information by testing larger images or simple text.

A major part of the project will be examining the tradeoff between time and information recovery. Since we assume the attacker can remain active for a long period, we can repeat measurements to reduce noise and improve confidence instead of focusing primarily on maximizing attack speed.

For our stretch goal, the malicious webpage will maintain a record of which portions of the victim content have already been tested and the confidence associated with each recovered value. This state could be stored by the malicious origin so that, if the user leaves and later returns, the experiment

can continue from the previous recovery state instead of starting again.

5 Evaluation

We will evaluate the attack primarily based on recovery accuracy and recovery time. Because the victim webpage is generated by us, we will know the correct contents and can directly compare the reconstructed result with the original.

We will measure how accurately individual pixels, regions, or characters can be inferred; how many measurements are required for a reliable inference; and how total recovery time grows as the victim content becomes more complicated.

6 Related Work

Our project is primarily motivated by "Pixel Thief: Exploiting SVG Filter Leakage in Firefox and Chrome" [1] as well as "GPU.zip: On the Side-Channel Implications of Hardware-Based Graphical Data Compression" [2]. These papers show that cache-based and memory-based side channel attacks can be used to leak information from a victim webpage, and will be the basis for our reproduction.

7 Conclusion

This project will investigate whether information displayed by a protected webpage can be indirectly reconstructed through browser rendering side channels. We will build a controlled malicious and victim webpage environment, attempt to recover known visual information, and evaluate the accuracy, reliability, and time required for recovery.

A complete reconstruction of arbitrary webpages is not required for the project to be successful. If the basic attack is successful early enough, we will explore more difficult scenarios and implement persistent recovery so that an attacker could continue collecting information across multiple visits to the malicious webpage.

References

- [1] O'CONNELL, S., SOUR, L. A., MAGEN, R., GENKIN, D., OREN, Y., SHACHAM, H., AND YAROM, Y. Pixel thief: Exploiting SVG filter leakage in firefox and chrome. In *33rd USENIX Security Symposium (USENIX Security 24)* (Philadelphia, PA, Aug. 2024), USENIX Association, pp. 3331–3348.
- [2] WANG, Y., PACCAGNELLA, R., GANG, Z., VASQUEZ, W. R., KOHLBRENNER, D., SHACHAM, H., AND FLETCHER, C. W. GPU.zip: On the Side-Channel Implications of Hardware-Based Graphical Data Compression . In *2024 IEEE Symposium on Security and Privacy (SP)* (Los Alamitos, CA, USA, May 2024), IEEE Computer Society, pp. 3716–3734.